```
PUT iphones/_doc/4
{
  "name":"iPhone 12 Mini",
  "release_date":"13-11-2020"
}
```

Now that we have an index containing several iPhones, let's develop a query to satisfy our requirement: we want to fetch all iPhones but prioritize those released 30 days around December 1, 2020. The query in the next listing does this.

Listing 12.23 Fetching iPhones and boosting the ratings

```
GET iphones/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "name": "12"
          }
        }
      ],
      "should": [
        {
          "distance_feature": {
            "field": "release_date",
            "origin": "1-12-2020",
            "pivot": "30 d"
          }
        }
      ]
    }
  }
}
```

The field against which our query executes

Defines the focal date

Pivots the number of days to boost scores

In this query, we wrap a `distance_feature` in a `bool` query with a `must` clause and a `should` clause (we discussed the `bool` query in chapter 11). The `must` clause searches for all documents with 12 in the `name` field and returns the iPhone 12 and iPhone 12 mini documents from our index. Our requirement is to prioritize phones released 30 days around the first of December (potentially, all phones released in November and December 2020).

To satisfy this requirement, the `should` clause uses the `distance_feature` query to enhance the scores of matching documents closest to the pivot date. The query fetches all the documents from the `iphones` index. Any iPhone released 30 days before or after December 1, 2020 (the origin) is returned with a higher relevance score.

Remember that the matches the `should` clause returns add to the overall score. So, the iPhone 12 Mini should top the list because the release date (`"release_date": "13-11-2020"`) of this iPhone is closer to the pivot date (`"origin":"01-12-2020"` ± 30 days). The results of the query are shown here for completeness: